For Loop in Python

09 November 2024 16:40

In Python, the for loop is used to iterate over sequences, such as list, tuple, strings and even **ranges**. Each item in the sequence is accessed one at a time, allowing you to perform operations on each element. Here's a comprehensive look at how the for loop works, along with examples:

Basic Structure of a for Loop:

Syntax:

```
for variable in range(N):
    #code to execute
```

"range() methods can be written in three ways:

1. range(stop): we provide a stop value, means the start is 0 and stop is always N-1. For example:

assigned to the variable one by one.

```
for i in range(15):
    print(i)
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]

i = 0 1 14

0
1
⋮
14

2. range(start, stop): here we had a facility to place the starting value and stop value. Here start < stop, otherwise range() method will not generate values. For example:

```
print('printing j')
for j in range(5, 10):
    print(j)
```

3. range(start, stop, step): By default the value of step is 1. Difference between each number in a sequence. For example:

Case of positive step value: start < stop

```
print('printing k')
for k in range(5, 15, 3):
    print(k)
```

Case of negative step value: start > stop [the generation of values will end just before the value of stop]:

For example:

```
print('printing h')
for h in range(15, 1, -3): #[15,12,9,6,3]
    print(h)
```

WAP to input number of terms and print the following series along with sum:

S = 2 - 4 + 6 - 8 + 10 - 12…

For example:

Input:

Number of terms: 5

Output:

2 - 4 + 6 - 8 + 10 = 6

Algebra of signs

+ × + = +
− × + = −
− × − = +
+ × − = −

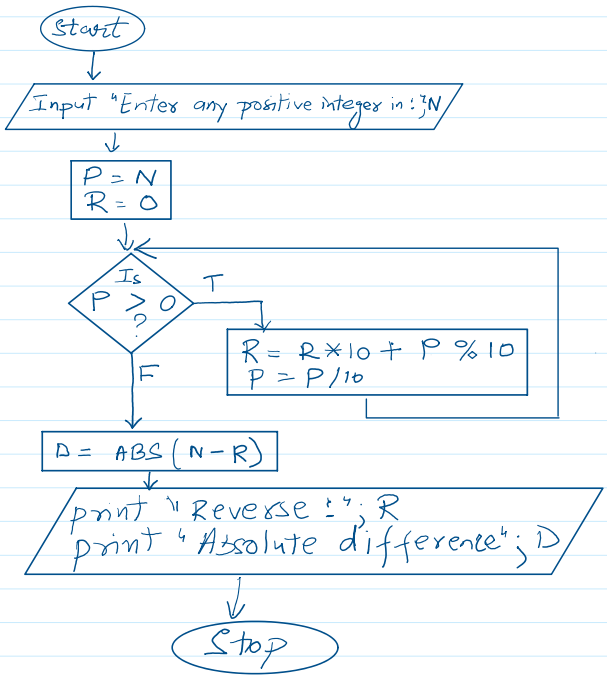
Solution:

```python
N = int(input('Number of terms: '))
sum = 0
sign = 1
for i in range(1, N):
    t = (i * 2)*sign
    if t > 0:
        print(t,sep='', end='')
    else:
        print(t,'+', sep='', end='')
    sum = sum + t
    sign = sign * (-1)
# for the last term
sum = sum + N*2*sign
print(N*2,'=',sum)
```

i = 1
while i < N

i = i + 1

1. Write a program to input and display the new number after reversing the digit of the original number. The program also displays the absolute difference between the original number and the reversed number.
   Input:
   Enter a number: 194
   Reverse: 491
   Absolute difference: abs(194 - 491) = 297
   [Hint:
   import math
   math.abs(194-491) = 297]



P = 194 ≠ 0

R = 491

4×10 + 9
49×10 + 1

2. In order to reach the top of a pole, a monkey in his first attempt reaches a height of 5 feet and in subsequent jumps he slips by 2% of the height attained in the previous jump. The process repeats. And finally, the monkey reaches the top of the pole. Write a program to input the height of the pole and calculate the number of attempts the monkey makes to reach the top of the pole.

3. WAP to check the number entered by the user is a palindrome or not. A number is a Palindrome which read in the reverse order is the same as original number itself. For example,
   Input:
   Enter a number: 246
   Output: Reverse: 642
   Not a palindrome number.

```
Input:
Enter a number: 242
Output: Reverse: 242
        Palindrome number.
```

4. WAP to input a number and check that number is perfect number of Not. A number is called Perfect if is equal to the sum of its factors other than the number itself.)
   Example: 6 = 1 + 2 + 3

```python
sum=0
i = 1
print(n," = ",end=" ")
output=""
while i <= n//2 :
    if (n % i==0) :
        output+=str(i)+" "
        sum+=i
    i = i + 1
num=output.split()
idx=0
while idx<len(num)-1:
    print(num[idx],end=" + ")
    idx+=1
print(num[idx])
if sum==n:
    print("\nPerfect number")
else:
    print("\nNot a perfect number")
```

i = ~~1~~, ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~, ~~7~~, ~~8~~, ~~9~~, ~~10~~, ~~11~~, ~~12~~, ~~13~~, ~~14~~, ~~15~~, ~~16~~, ~~17~~
n = 35

output = 1_5_7_

sum = ~~1~~ ~~5~~ 13

num → [1 | 5 | 7]

'len' function counts how many boxes.

= 1 + 5 + 7

⇒ Strings

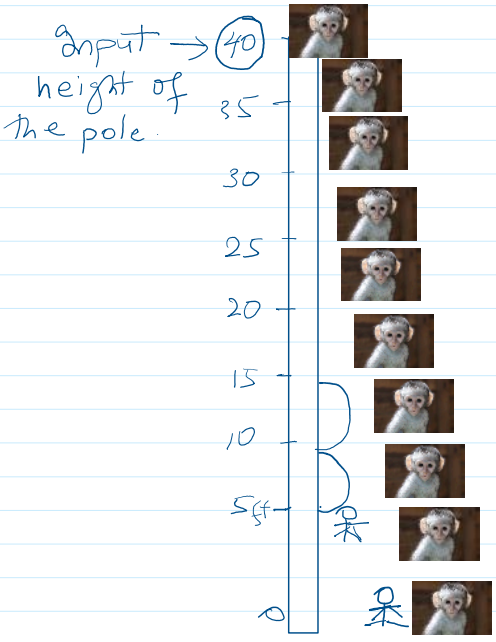
⇒ Lists

// ← quotient

% ← remainder

```
17
2 | 35
  | 2
  15
  14
   1
```

In order to reach the top of a pole, a monkey in his first attempt reaches a height of 5-feet and in subsequent jumps he slips by 2% of the height attained in the previous jump. The process repeats. And finally, the monkey reaches the top of the pole. Write a program to input the height of the pole and calculate the number of attempts the monkey makes to reach the top of the pole.



$5 - \frac{1}{10}$ $\frac{49}{10}$

1. WAP to input a number and check that number is automorphic or not. An automorphic number is the number which contained in the last digits(s) of its square. For example:
   Input: Enter any number: 5
   Output: Automorphic number

   Input: Enter any number: 25
   Output: Automorphic number

   Input: Enter any number: 7
   Output: Not Automorphic number

Home Work:

2. A special two-digit number is such when the sum of its digits is added to the product of its digits, the result is equal to the original two-digit number.
   Example: Consider the number 59.
   Sum of digits = 5 + 9 = 14
   Product of digits = 5 * 9 = 45
   Sum of the sum of digits and product of digits = 14 + 45 = 59

   Write a program to accept a two-digit number. Add the sum of its digits to the product of its digits. If the value is equal to the number input, then display the message "Special 2 two-digit number".

3. Write a program to input a number. Check and display whether it is a Niven number or not. (A number is said to be Niven when it is divisible by the sum of its digits).
   Sample Input: 126
   Sum of its digits = 1 + 2 + 6 = 9 and 126 is divisible by 9.

4. Write a program to accept a number and check whether it is a 'Spy Number' or not. (A number is a spy if the sum of its digits equals the product of its digits.)
   Sample Input: 1124
   Sum of the digits = 1 + 1 + 2 + 4 = 8
   Product of the digits = 1*1*2*4 = 8
   Hence 1124 is a Spy number.

5. Write a program to display the Mathematical Table from 5 to 10 for 10 iterations in the given format:
   Sample Output: Table of 5
   5*1 = 5
   5*2 = 10
   5*3 = 15
   …
   …
   5*10 = 50

   And so on…

```
num=int(input("Enter any number"))
square=num**2
while num>0:
    rem=square%10
    n=num%10
    if rem!=n:
        break
    square//=10
    num//=10
if num==0:
    print("Automorphic number")
else:
    print("Not an automorphic number")
```

num = ~~25~~ 0
square = ~~625~~
rem = ~~5~~ 2
n = ~~5~~ 2

num = 17
square = 289
rem = 9
n = 7

quotient

```
      62
10 | 625 |
     60
     ---
      25
      20
      --
       5
```

remainder